

Notes and updates on Action Plans

Kurt Biery, on behalf of Eric Flumerfelt and others

22 July 2025

Software Integration & Testing meeting

Introduction: why is this relevant now?

As people are adding new Applications to configurations, if they copy/paste an existing App declaration for their new App, there is a chance that the App module list gets out of sync with Action Plan module lists. We saw this with CRT Apps, and we see that it has been an existing issue in other places in our sample configuration.

Also, it turns out that the handling of Action Plans in the code and configuration is not as robust or easy as we would like.

So, there is value in fixing the issues that we have found and providing some background information that we can use to help when thinking about improvements.

[part of my motivation in writing these slides is to provide additional documentation on Action Plans...]

Why would a developer/user care...

...about Action Plans?

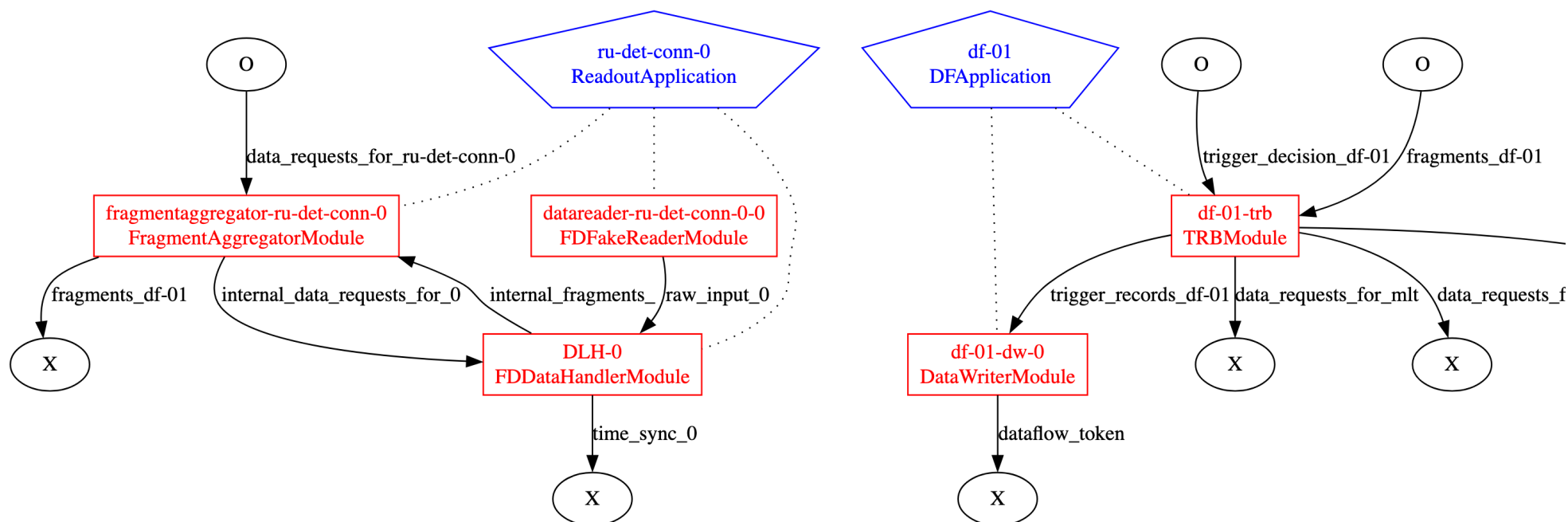
Because they are something that we need to get right when we modify configurations, especially adding new Applications.

And, if you are adding a new App, you may need to think about the order of operations within your App.

The problem that Action Plans solve...

Specifying the order in which transitions happen at the DAQModule level (inside DAQApplications)...

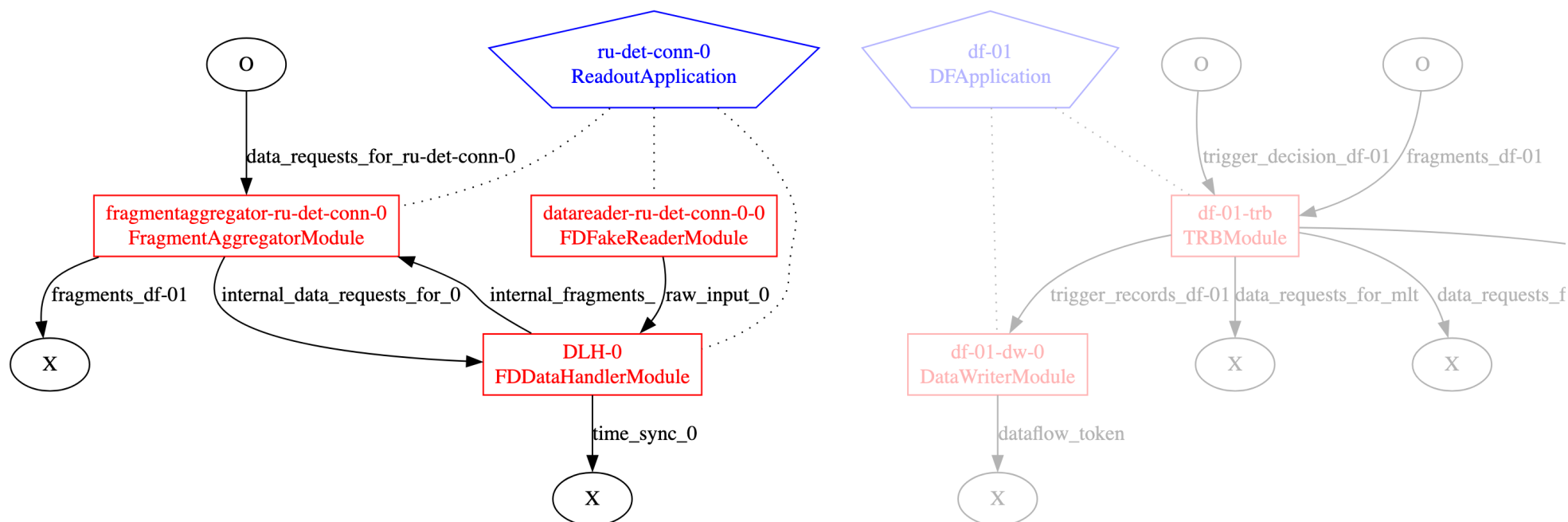
Consider *start* (starting a run) in simple Readout and Dataflow apps (*taken from small_footprint_quick_test*)...



Order of *start* in a simple Readout App

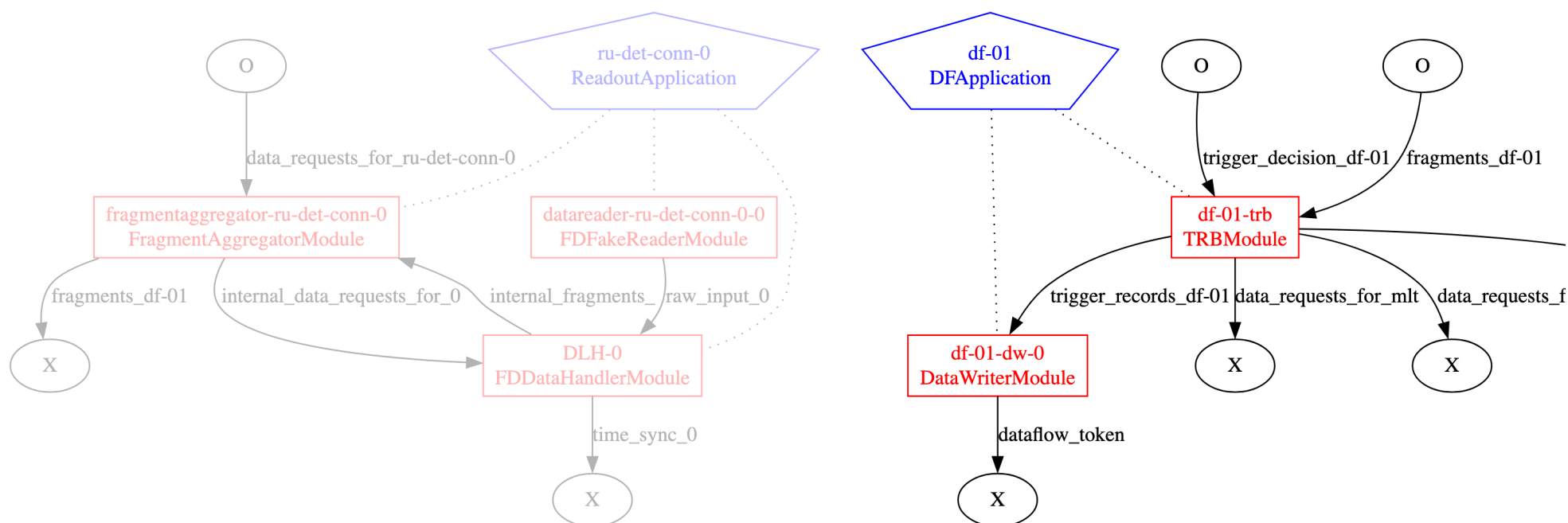
Executing the *start* command in the *FDFakeReaderModule* before the *FDDataHandlerModule* could cause problems because the *raw_input_0* queue could fill up.

- And, sending the *start* command to both of them in parallel could result in the wrong order...



Order of *start* in a simple Dataflow App

Since TriggerRecords are not being built in the DAQ system until after the *enable-triggers* transition (which comes after *start*), there is no risk in sending the *start* command to the *TRBModule* and the *DataWriterModule* in parallel. We could do it in a specified order, but that would add a maintenance burden to our future selves with no obvious benefit.



Action Plans are how we do this

Action Plans are

- Specified in our configurations
- “a series of steps consisting of groups of modules, which are executed in response to a command from CCM.” ([doc link](#))

An example: (loosely copied from daqsystemtest/config/daqsystemtest/moduleconfs.data.xml)

```
<obj class="ActionPlan" id="readout-start">
  <attr name="execution_policy" type="enum" val="modules-in-parallel"/>
  <rel name="command" class="FSMCommand" id="start"/>
  <rel name="steps">
    <ref class="DaqModulesGroupByType" id="aggregator-step"/>
    <ref class="DaqModulesGroupByType" id="dlh-step"/>
    <ref class="DaqModulesGroupByType" id="data-source-step"/>
  </rel>
</obj>
```

- This may raise more questions that it answers, but hopefully things will become more clear as we go along.

Quick side note, fixed vs. flexible text

Often when I see a snippet of one of our configurations, it is not immediately clear to me which parts are “fixed” (that is, the text needs to appear exactly as shown) and which parts are flexible (that is, the names could be changed by me without causing problems).

To help make this clear in this talk, I’ll colorize the “fixed” parts in gray text, the pick-from-existing-values parts in blue text, and the fairly free-form parts in green text.

```
<obj class="ActionPlan" id="readout-start">
  <attr name="execution_policy" type="enum" val="modules-in-parallel"/>
  <rel name="command" class="FSMCommand" id="start"/>
  <rel name="steps">
    <ref class="DaqModulesGroupByType" id="aggregator-step"/>
    <ref class="DaqModulesGroupByType" id="dlh-step"/>
    <ref class="DaqModulesGroupByType" id="data-source-step"/>
  </rel>
</obj>
```

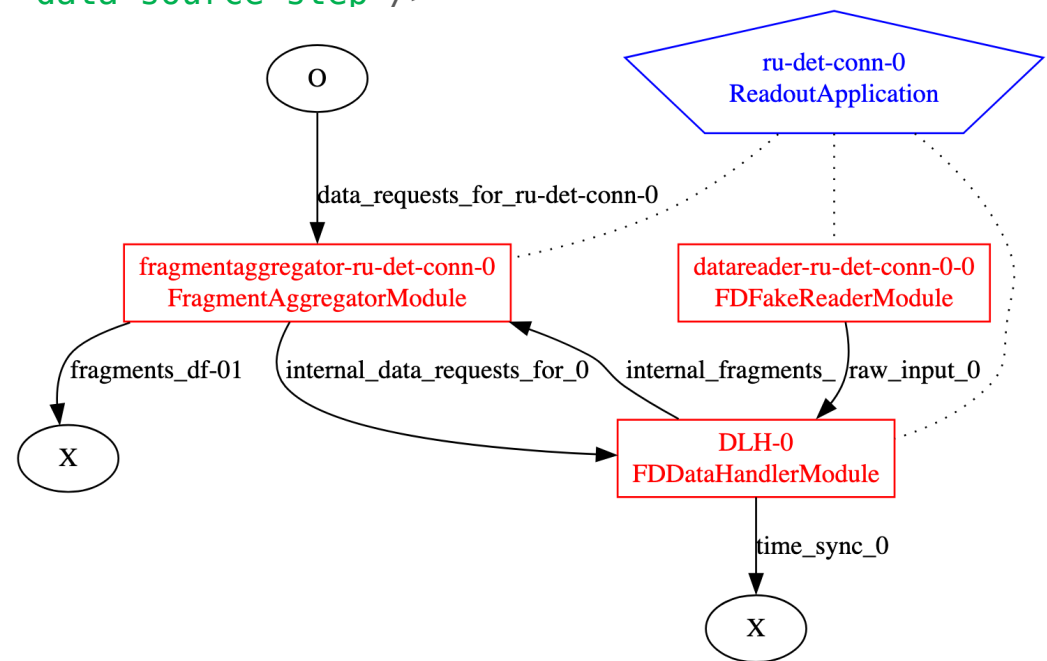

A quick touch-point

Comparing the example Action Plan shown two slides ago to the simple Readout App diagram that we looked at earlier...

```
<obj class="ActionPlan" id="readout-start">
  <attr name="execution_policy" type="enum" val="modules-in-parallel"/>
  <rel name="command" class="FSMCommand" id="start"/>
  <rel name="steps">
    <ref class="DaqModulesGroupByType" id="aggregator-step"/>
    <ref class="DaqModulesGroupByType" id="dlh-step"/>
    <ref class="DaqModulesGroupByType" id="data-source-step"/>
  </rel>
</obj>
```

hopefully, we can start to see the pattern...

- aggregator-step ~ FragAggModule
- dlh-step ~ FDDataHandlerModule
- data-source-step ~ FDFakeRdrModule



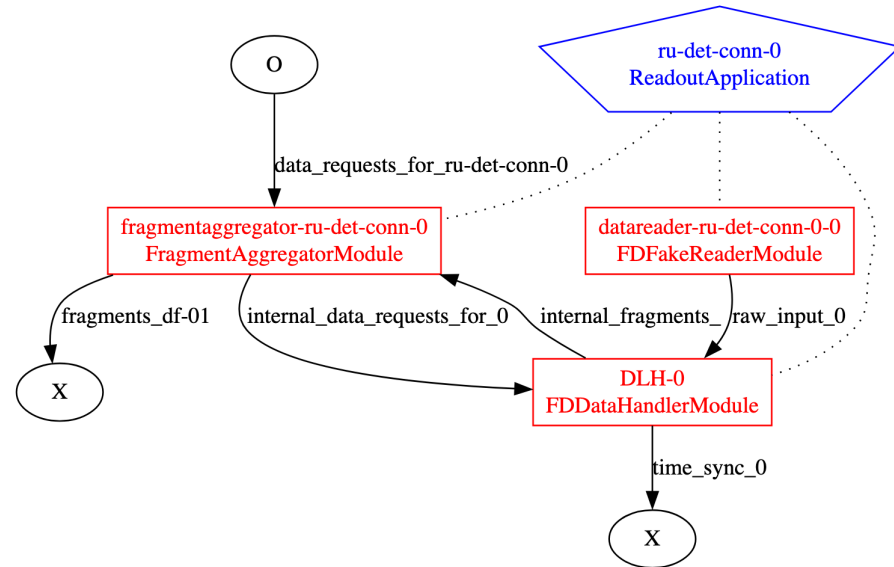
Some notable caveats

1. If no Action Plan is specified for the modules within a DAQApplication for a given transition, then that transition is executed in all of the modules in parallel (with them receiving the command in random** order).
 1. “Random” isn’t actually correct. The order that is used is the order that the modules are listed in the DAQApplication configuration entry.
2. If an Action Plan is specified for the modules within a DAQApplication for a given transition, then all modules need to appear somewhere in the Action Plan.
 1. If they are not listed, then they don’t receive the command.
3. *Steps* in an Action Plan are always executed in series.

Caveat 1: default action plan (parallel)

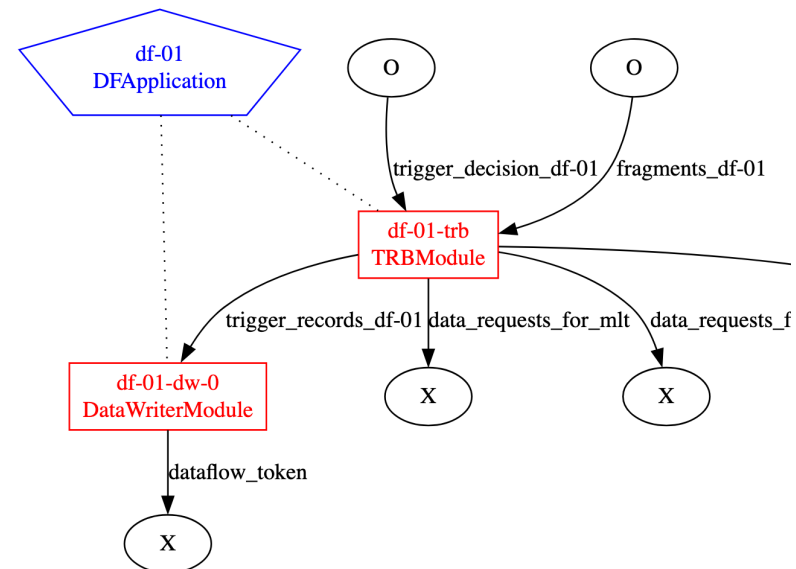
For the Readout App, all three modules would receive *start* at essentially the same time

- This would be risky



For the Dataflow App, both modules would receive *start* at the same time

- This seems like a very reasonable choice here



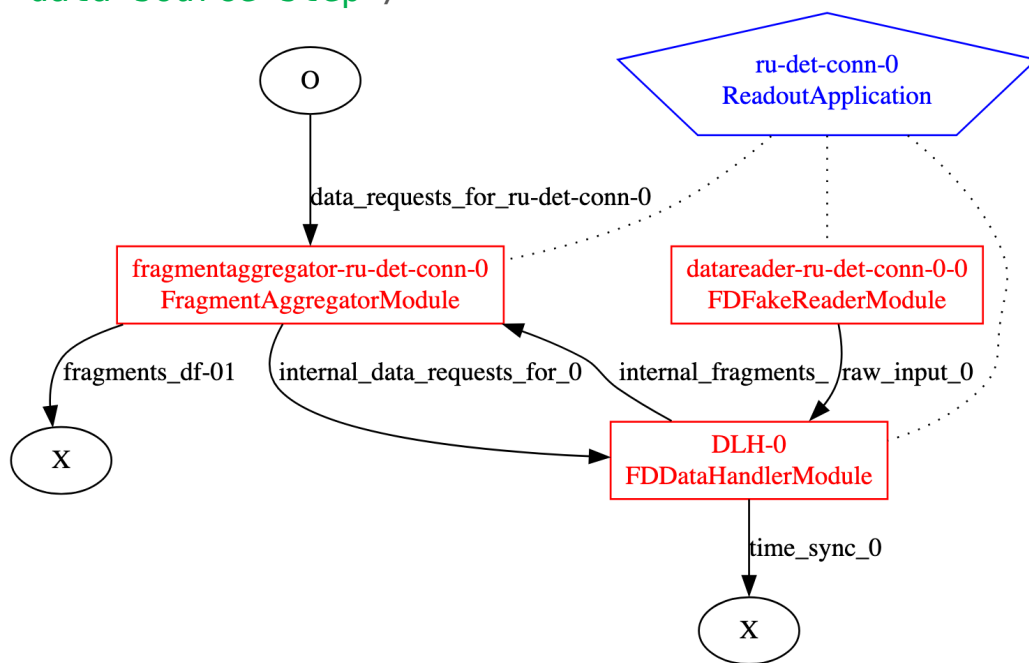
Caveat 2: all modules must be listed in plan

The example Action Plan that we saw earlier looked like it handled this correctly...

```
<obj class="ActionPlan" id="readout-start">
  <attr name="execution_policy" type="enum" val="modules-in-parallel"/>
  <rel name="command" class="FSMCommand" id="start"/>
  <rel name="steps">
    <ref class="DaqModulesGroupByType" id="aggregator-step"/>
    <ref class="DaqModulesGroupByType" id="dlh-step"/>
    <ref class="DaqModulesGroupByType" id="data-source-step"/>
  </rel>
</obj>
```

assuming...

- aggregator-step ~ FragAggModule
- dlh-step ~ FDDataHandlerModule
- data-source-step ~ FDFakeRdrModule



Caveat 3: steps executed in series

I was recently confused by the *execution_policy* value of "modules_in_parallel" that we saw in the example Plan:

```
<obj class="ActionPlan" id="readout-start">
  <attr name="execution_policy" type="enum" val="modules-in-parallel"/>
  <rel name="command" class="FSMCommand" id="start"/>
  <rel name="steps">
    <ref class="DaqModulesGroupByType" id="aggregator-step"/>
    <ref class="DaqModulesGroupByType" id="dlh-step"/>
    <ref class="DaqModulesGroupByType" id="data-source-step"/>
  </rel>
</obj>
```

But, that is referring to the modules within each step.

So, how would we write an Action Plan that reproduces the default behavior that we get when no Action Plan is specified?

- Create an Action Plan with one step
- List all of the modules in that single step
- Set the *execution_policy* to "modules-in-parallel"

More details on configuration, part 1

I've glossed over some of the details of specifying Action Plans in our configurations when I just showed this:

```
<obj class="ActionPlan" id="readout-start">
  <attr name="execution_policy" type="enum" val="modules-in-parallel"/>
  <rel name="command" class="FSMCommand" id="start"/>
  <rel name="steps">
    <ref class="DaqModulesGroupByType" id="aggregator-step"/>
    <ref class="DaqModulesGroupByType" id="dlh-step"/>
    <ref class="DaqModulesGroupByType" id="data-source-step"/>
  </rel>
</obj>
```

You may be wondering, “what is in aggregator-step, etc?” Here are some examples:

```
<obj class="DaqModulesGroupByType" id="aggregator-step">
  <attr name="modules" type="class">
    <data val="FragmentAggregatorModule"/>
  </attr>
</obj>
<obj class="DaqModulesGroupByType" id="dlh-step">
  <attr name="modules" type="class">
    <data val="FDDataHandlerModule"/>
  </attr>
</obj>
```

Hopefully, this makes sense, given what we assumed earlier...

More details on configuration, part 2

You may also be wondering, “how would I add an Action Plan to an Application?” Here is an example, with some config text suppressed, and different colorization...

```
<obj class="ReadoutApplication" id="ru-det-conn-0">
  <attr name="application_name" type="string" val="daq_application"/>
  <attr name="tp_generation_enabled" type="bool" val="0"/>
  <attr name="ta_generation_enabled" type="bool" val="0"/>
  <rel name="contains">
    <ref class="DetectorToDaqConnection" id="det-conn-0"/>
  </rel>
  <!-- entries removed to help readability -->
  <rel name="network_rules">
    <ref class="NetworkConnectionRule" id="data-req-readout-net-rule"/> <!-- FragmentAggregator -->
    <ref class="NetworkConnectionRule" id="tpset-net-rule"/>
    <ref class="NetworkConnectionRule" id="ts-net-rule"/>
    <ref class="NetworkConnectionRule" id="ta-net-rule"/>
  </rel>
  <rel name="action_plans">
    <ref class="ActionPlan" id="readout-start"/>
    <ref class="ActionPlan" id="readout-stop"/>
  </rel>
  <rel name="uses" class="RoHwConfig" id="rohw-det-conn-0"/>
  <rel name="link_handler" class="DataHandlerConf" id="def-link-handler"/> <!-- FDDataHandlerModule -->
  <rel name="data_reader" class="DPDKReaderConf" id="nicrcvr-fake-gen"/> <!-- FDFakeReaderModule -->
</obj>
```

Things that we need to talk about...

1. What happens if a module is listed in an Action Plan but is not listed in the Application?
 - In fddaq-v5.3.2 and earlier, this would cause an ERS INFO message to be written to the application log file.
 - Eric has prepared code changes in which this generates an ERS WARNING.
2. What happens if a modules is listed in an Application but not in the Action Plan?
 - In fddaq-v5.3.2 and earlier, the transition command would silently not be delivered to the DAQModule.
 - Eric has prepared code changes in which this generates an ERS ERROR, in addition to not being delivered to the DAQModule.
3. Flexible and/or future ways of specifying Action Plans
 - Next slide...

Flexible/future Action Plans

Eric and I talked briefly about catch-all logic and wildcard possibilities, but we didn't go very far down that path.

Independent of that, Alessandro pointed out that it would be ideal if we didn't need to separately specify the modules in an Application and the modules in the Action Plan (and ensure that they are kept in sync) [[link](#)]. Hopefully, something along these lines will become available someday.

Practical considerations for today

I propose that our first step is to review Eric's changes for ERS reporting (with new unit tests) and get those merged into *develop*. Also, we can update our example configs (e.g. in *daqsystemtest/example_configs.data.xml*) to take into account the known boundary conditions. (These are on the *kbiery/readout_tpg_action_plans* branches in *daqconf* and *daqsystemtest*.)

We should talk about what may need to change in *ehn1-configs* when we start reporting more errors and warnings about Action Plans.

Appendix A: modules types vs. IDs

I'll come back to this next week.

Appendix B: addl. examples

For more complicated systems...